

背景

1.背景:

“水木汇”作为清华校园的活动报名小程序，在院系组织发起的抢票类活动（限额报名、先到先得）下，前端会在短时间内触达大量并发用户。当前后端为 Node.js + MongoDB，前端为微信小程序，单台数据库的原子写成为瓶颈（约 20 条/秒），前端频繁拉取活动介绍图片又给服务器和网络造成压力。为了保证用户体验与平台稳定性，需在不完全更换硬件的前提下，通过架构优化将系统改造为“前端快速响应 + 后端异步持久化”的模式。

2.需求:

为微信小程序“水木汇”提供一套可执行的高并发改造方案，目标在现有学校租用虚拟机（Westmere 16 核/32GB/TopHC vHDD）条件下，将并发能力从当前瓶颈（DB原子写 20 tps）提升到 **5k-10k 并发请求处理能力**（注意：并发连接与后端可持续TPS是两个度量，本文中目标为能在抢票场景下短时间内并发接收并快速返回结果，最终数据库写入将异步化，能够稳定承受峰值压力）。

改进点

Redis

进行抢票服务的原子抢占，后续再异步入库（mongodb）

后端重构

后端重构为go

- 如果你的系统**瓶颈确实在 Node.js 的并发与 GC 开销**，迁移到 Go **理论上可以提升 5-10 倍性能**；
- 但如果瓶颈是数据库写入或架构设计（例如同步写 MongoDB），**换语言几乎无效**；
- 对于“抢票场景”，**Go 更合适**，因为它能承受突发高并发并快速返回响应。
- 中长期方案

反向代理

使用nginx进行反向代理

- 对静态文件进行分流
- 开启静态文件压缩与缓存

oss组件

若静态资源有鉴权逻辑，则引入oss组件存储静态文件，对同样的缓存授权连接处理，利用顺序读写能力得到读写性能提升

- 不必要，若静态图片数量庞大或有鉴权需求，则考虑
- nginx+本地存储+signed URLs可作为简单解决方案

调大文件描述符

linux系统，若服务器未经优化，可调大文件描述符提升性能

k3s

利用容器动态扩容，在单机场景下作用有限

mongodb

优化处理和存储逻辑

架构考虑点

- **负载均衡 / 反向代理** (nginx / traefik) : SSL、连接复用、静态缓存、速率限制入口。
- **应用层 (Node.js)** : 若干无状态实例，负责接收请求、做快速原子判断 (参见 Redis Lua) , 将“成功预占/抢票结果 + 事件”入队，然后立即响应前端 (成功/失败) 。短请求时延。
- **Redis (主内存缓存 + 队列/流)** :
 - 用作抢票的**计数器/库存管理** (原子递减由 Lua 脚本完成) 。
 - 用作**轻量队列/Stream** 保存事件，用于后端异步持久化。
 - 也可用于幂等/限流 (token bucket) 、黑名单与防刷。
- **后台工作进程 (Worker, Node.js 或 Go)** :
 - 从 Redis Stream (或 List) 消费事件，做**批量写入 MongoDB** (bulkWrite, ordered: false) 。
 - 失败重试、死信队列、告警。
- **MongoDB (现有)** :
 - 保持为最终持久化存储，写入策略调整为异步批量写、较低写一致性 (w:1 可选) 以换吞吐。
- **静态资源/图片**:
 - 抽取并迁移到 CDN 或对象存储 (如腾讯 COS / 阿里 OSS / 七牛) , 并设置 aggressive 缓存 (Cache-Control) 。
 - 小程序端使用本地缓存、适当压缩图片、懒加载。
- **监控与告警**: Prometheus + Grafana 或直接云监控，采集 Redis 队列长度、请求延迟、后端批量写吞吐、错误率、qps。
- **限流 & 防刷**: 请求入口 (nginx + Redis) 做漏桶/令牌桶限流 + 验证码/行为风控策略。

基本技术

k3s

K3s 是一个经过优化和精简的 Kubernetes 发行版

特点:

- **轻量级**: 所有组件都打包成一个小于 100MB 的二进制文件，启动速度快，内存占用极低。
- **简化安装**: 只需一条命令即可完成一个节点的安装和配置，大大降低了入门和运维复杂度。
- **低资源消耗**: 非常适合在边缘计算、物联网设备、开发机、ARM 架构设备 (如树莓派) 上运行。
- **兼容性**: 100% 兼容原生 Kubernetes 的 API。这意味着为原生 Kubernetes 编写的 YAML 配置文件和应用，通常可以无缝地在 K3s 上运行。

- **内置组件：** 它用轻量级的替代品替换了 Kubernetes 的一些资源密集型组件（如用 SQLite 替代 etcd 作为默认存储），并内置了常用的组件（如 Helm controller, Traefik Ingress controller），开箱即用。

OSS

对象存储服务，分为云存储方案如腾讯云等和本地方案如minIO

日程计划

第一阶段：需求分析与架构设计（第 1 周）

目标：

- 明确系统当前瓶颈，分析现有代码结构。
- 设计新的架构方案，确保能在现有硬件条件下实现高并发。

任务：

1. 需求确认与背景分析

- 理解现有系统的架构。
- 分析现有瓶颈。

2. 系统架构设计

- 设计高并发场景下的架构优化方案。
- 确定 Redis 的使用，作为抢票计数与预占缓存。
- 设计异步队列，使用 Redis 或 Kafka 实现异步持久化。
- 确定图片资源优化策略（nginx 静态资源缓存、压缩等）。

3. 技术栈学习与工具准备

- 学习 Redis Lua 脚本、MongoDB 批量写入、nginx 配置。
- 搭建开发环境并确保系统可以在虚拟机上运行（Node.js / Go / MongoDB / Redis）。

4. 初步代码审查与改进计划

- 审查现有代码，确认业务逻辑与数据库访问路径。
- 确定代码重构与优化点，制定改进计划。

第二阶段：核心功能实现（第 2 周）

目标：

- 实现抢票核心功能，并优化数据库操作。
- 完成 Redis 预占与库存更新逻辑，确保高并发时能快速响应。

任务：

1. Redis 实现库存预占与抢票逻辑

- 编写 Redis Lua 脚本实现抢票的原子操作（库存扣减、预占状态更新）。
- 实现 Redis 的预占缓存（存储用户预占的票）。

2. 异步队列设计与实现

- 使用 Redis List 或 Stream 实现异步队列。
- 将抢票成功的请求推送到队列中，等待后台异步写入数据库。

3. MongoDB 批量写入优化

- 改写数据库写入逻辑，使用 MongoDB 的 `bulkwrite` 操作提高写入性能。
- 实现批量数据写入与幂等性，避免数据丢失。

4. 前端与后端接口联调

- 实现前端与后端的通信接口，确保前端能够快速响应并反馈抢票结果。
- 在前端显示“预占成功/失败”提示，后端返回抢票状态。

5. 单元测试与基本功能验证

- 编写单元测试，确保 Redis 预占、队列推送、MongoDB 写入等核心功能的正确性。
- 测试数据库写入时的并发情况，确保数据一致性。

第三阶段：性能优化与压力测试（第 3 周）

目标：

- 优化系统性能，确保能够处理高并发请求。
- 进行压力测试，发现瓶颈并调整系统配置。

任务：

1. 系统性能优化

- 配置 Redis、MongoDB、nginx 等系统参数，确保高并发时系统能够平稳运行。
- 调整 Redis 连接池、MongoDB 批量写入参数等。

2. nginx 配置与静态资源优化

- 配置 nginx 实现静态资源的高效缓存，启用 gzip 压缩。
- 优化静态图片加载路径与缓存策略，减少前端对服务器的频繁请求。

3. 压力测试与性能调优

- 使用压力测试工具（如 wrk、Locust）进行高并发压测，模拟真实用户请求。
- 测试 Redis 和 MongoDB 的负载能力，找出性能瓶颈。
- 通过日志和监控工具分析系统响应时间与资源占用情况，进行调优。

4. 并发场景验证与系统稳定性测试

- 验证高并发场景下的库存预占、用户抢票、队列写入和数据库持久化等功能。
- 对 Redis、MongoDB 进行负载测试，确保数据处理与存储稳定。

第四阶段：系统集成与部署（第 4 周）

目标：

- 完成系统的集成与部署，确保系统在生产环境中稳定运行。
- 提交项目并进行最终的性能验证与文档编写。

任务：

1. 集成与测试

- 集成前端与后端，确保整个抢票流程顺畅。
- 进行全流程测试，包括抢票、预占、后台持久化、前端反馈等。
- 调试并修复集成过程中出现的 bug。

2. 部署与上线准备

- 部署应用到生产环境，配置 nginx、Redis 和 MongoDB。
- 配置负载均衡与容灾方案，确保系统稳定性。
- 进行生产环境的性能验证，确保系统能够承受高并发压力。

3. 项目文档与报告编写

- 编写项目文档，详细描述系统架构、技术实现、优化方案与性能测试结果。
- 撰写项目总结报告，介绍项目目标、技术难点与解决方案。

4. 最终检查与交付

- 进行最后的系统检查，确保没有遗漏的功能或错误。
- 提交项目代码，交付项目成果，并进行项目演示。

总结与后期工作

后期优化：

- 根据生产环境中的反馈，对系统进行持续优化。
- 若流量持续增长，可考虑将部分服务迁移到更强大的硬件或云端，或采用水平扩展方案（如 Redis 集群、MongoDB 分片）。
- 持续监控系统性能，处理潜在的性能瓶颈与故障恢复。